

Vector field for curve tracking with obstacle avoidance

Arthur H. D. Nunes, Adriano M. C. Rezende, Gilmar P. Cruz Junior,
Gustavo M. Freitas, Vinicius M. Gonçalves, and Luciano C. A. Pimenta

Abstract—In this work, we extend a recently proposed methodology to construct artificial vector fields for robot navigation in n -dimensional spaces to track and circulate time-varying curves. Now, we incorporate the ability to deviate from obstacles that might be static or dynamic by constructing a collision-free vector field. It can be considered a reactive approach in which the obstacles can be locally sensed and then circumnavigated at a fixed distance. The novel vector field that allows the circumnavigation of obstacles and the traversal of a target curve is built upon the consideration of the closest points on the obstacles, which can assume a generic shape, and the closest point on the curve. To validate our method, we present ROS-based computational simulations and real-world experiments with a wheeled robot and a UAV quadcopter.

I. INTRODUCTION

The autonomous robot navigation problem is an important challenge in the field of robotics that aims to make an autonomous robot move safely in a given environment filled with obstacles. The solution to this problem can be applied to many applications such as surveillance, inspection, search and rescue, among others.

Artificial vector field strategies have been widely used for robot navigation. In this work, we extend a recently proposed vector field method for robot navigation, [1], to incorporate an obstacle avoidance feature, enabling robots to freely navigate through a workspace with obstacles that are not known a priori and might even be dynamic. The extension proposed here is based on the modification of the original field to allow not only the traversal of time-varying target curves but also obstacle-circumnavigation at a fixed distance, which can be locally computed.

A. Artificial vector field for robot navigation

A strategy for robot navigation based on an artificial vector field that drives the robot to time-varying paths in n -dimensional spaces is proposed in [2]. The idea in that work is to implicitly define the target path by the intersection of zero level sets of scalar functions $\alpha_i : \mathbb{R}^n \times \mathbb{R}_+ \rightarrow \mathbb{R}$, i.e.

The authors are with the Graduate Program in Electrical Engineering (PPGEE) of the Universidade Federal de Minas Gerais (UFMG), Belo Horizonte, MG, 31270-901, Brazil (emails: arthurhdn@ufmg.br; adrianomcr@ufmg.br; gilmarpcjunior@ufmg.br; gustavomfreitas@ufmg.br; mariano@cpdee.ufmg.br; lucpim@cpdee.ufmg.br).

This work was in part supported by the project INCT (National Institute of Science and Technology) under the grant CNPq (Brazilian National Research Council) 465755/2014-3, FAPESP (São Paulo Research Foundation) 2014/50851-0, and FAPEMIG (grant numbers APQ-02144-18 and APQ-02228-22). This work was also supported in part by the Coordenação de Aperfeiçoamento de Pessoal de Nível Superior - Brasil (CAPES) - Finance Codes 88887.136349/2017-00 and 001, and Conselho Nacional de Desenvolvimento Científico e Tecnológico (CNPq) - grant numbers 315258/2020-9, 306286/2020-3, and 407063/2021-8.

$\cap \alpha_i = 0$. The authors validate the method in [3] with real robot experiments. This method is also applied in many other works such as [4], [5]. Other works also use similar vector field based strategies [6], [7].

The recent work in [8] presents a vector field methodology for n dimensions and static curves represented by a parametric equation. The methodology consists of an extension of the n -dimensional space with another dimension, which is correspondent to the parameter of the curve's representation. The authors define zero-level sets in this extended space of $n + 1$ dimensions and apply the theory proposed in [2] to create the vector fields. An important feature of this approach in extended spaces is that the resulting vector field is singularity-free. In addition, the methodology can be applied to curves with self-intersections, since in the extended dimension the curve does not intersect itself. However, they do not consider time-varying curves or presence of obstacles.

A new artificial vector field methodology for robot navigation in time-varying paths embedded in n -dimensional spaces was developed in our recent work [1]. In this recent method, the field is obtained constructively with the use of the Euclidean distance function to the curve. This field is not a gradient field of a potential function, since it induces a limit cycle for a closed curve [9]. It uses a parametric representation of the target curve, an advantage over the intersection of zero level sets required in [2]. In [1] we point out several characteristics and advantages of the new approach providing formal proofs. However, the presence of obstacles was not considered in [1].

B. Obstacle avoidance in robot navigation

A recurrent issue in the robot navigation problem is the presence of obstacles. In general, the methodologies that tackle the presence of obstacles in the robot's workspace consider approaches that can be classified either as reactive or deliberative.

A reactive approach considers navigation free of obstacles in the first moment and then actuates to avoid a collision when an obstacle appears. This obstacle is detected by an exteroceptive sensor in general. A deliberative approach considers some path planning that computes a path free of collisions. This planning usually requires the knowledge of the map and the geometry of the obstacles. An advantage of a reactive approach is that it is usually simple, so it can be easily applied to unknown environments and has a low computational cost. However, it may lead the robot to undesirable conditions and generate non-smooth paths.

A simple and widely known reactive strategy is the repulsive field action [10]. If this repulsive action points in the opposite direction of the original reference and cancels it, the robot stops its movement. Since this methodology originated from a scalar potential field, we usually say that the robot is stuck in a local minimum of this field.

With a primary objective of reaching a single goal point, the Bug algorithms [11] are also good examples of navigation algorithms that incorporate the ability to deviate from obstacles in two dimensions. The deviation strategy consists of the circumnavigation of the detected obstacles. The adopted approach is then reactive.

Other obstacle avoidance strategies have been studied when the primary objective of navigation is to follow a path. In [12] the authors consider a vector field together with a Rapidly-exploring Random Tree (RRT) to guide a fixed-wing UAV toward a curve while deviating from obstacles. In fact, the authors use the Vector Field Rapidly-exploring Random Tree (VFRRT) [13] to grow the tree with the definition of a path that deviates from the obstacles. This algorithm uses the original vector to influence the growth direction of the tree.

Adopting a similar idea of [12], in [14] the authors use the RRT*, cost optimum version of RRT, together with a vector field approach to generate optimal collision-free motion plans. Based on [14], the work [15] uses the RRT* with vector field to drive a semi-autonomous robot in the execution of user-defined movements. The RRT* is grown to maximize the alignment of the path with the field's trajectories, or integral lines. When obstacles are in the way, the planner generates a smooth path that attempts to follow the vector field as closely as the obstacle set allows. Although being a deliberative approach, the methodology presented in [15] does not require global localization, but only a local map and the detection of obstacles.

The works [16], [17] consist of an experimental evaluation of a method for path following with collision avoidance that deforms the original planned path to avoid modeled obstacles in a two-dimensional scenario with multiple robots. In [18], a gradient vector field (GVF) was used with a repulsive vector field to allow UAVs to perform circular movements around obstacles.

An integrated path following and collision avoidance method using a composite vector field is proposed in [19]. The authors use a static composite vector field with two components: one to follow the path and the other to deviate from obstacles. The former is based on zero-level sets as in [2] and the latter is obtained via bump functions. The idea is similar to the one presented in our work, but we have the advantages: (i) we allow obstacles to be generic-shaped; (ii) we consider dynamic obstacles; (iii) we use only the closest point in the obstacle for the deviation.

In [5] the authors consider a reactive approach that allows a fixed-wing UAV to deviate from moving obstacles when it is following a vector field that converges to a path. In that work, the authors also use an online switching vector field similar to the one presented here. However, the present work

has the following advantages: (i) it can deal with generic-shaped obstacles; (ii) it is suitable for n dimensions; (iii) it is validated in realistic simulations and real robots experiments; (iv) it extends the novel method developed in [1] that has advantages when compared to [2], which is used in [5].

The method in [1] considers the closest point on the path to compute the vector field. Here we will show how to apply this method to circumnavigate an obstacle by using only the closest obstacle point. This connection is an extra motivation for the use of the Euclidean vector field proposed in [1]. Next, we incorporate obstacle circumnavigation into the original field to generate a collision-free navigation strategy. We emphasize that this collision avoidance is not a potential field approach [9].

II. COLLISION-FREE VECTOR FIELD

In order to present our methodology, we first review the artificial vector field method proposed in [1]. Then, we use this field to develop a simple way to circumnavigate an obstacle. Finally, we present a novel collision-free vector field by extending the previously proposed method with the incorporation of the simple circumnavigation strategy.

A. Constructive artificial vector field review

To review the method in [1], we consider the first order integrator model given by

$$\dot{\mathbf{x}} = \mathbf{u}, \quad (1)$$

where $\mathbf{x} \in \mathbb{R}^n$ is the configuration vector to be controlled and $\mathbf{u} \in \mathbb{R}^n$ the configuration's velocity considered as the system input.

The goal is to find $\Phi(\mathbf{x}, t) : \mathbb{R}^n \times \mathbb{R}_+ \rightarrow \mathbb{R}^n$ such that if $\mathbf{u} = \Phi(\mathbf{x}, t)$ the model in (1) converges to and follows a time-varying curve $\mathcal{C}(t) \subset \mathbb{R}^n$ that is either an open curve, unbounded curve homeomorphic to a straight line, or a closed curve, a bounded curve homeomorphic to a circle. Let $\mathbf{r}(s, t) : \mathbb{R} \times \mathbb{R}_+ \rightarrow \mathbb{R}^n$ be a parametric representation for $\mathcal{C}(t)$, with s being the arc length and t the time and in which it is twice differentiable in s for all t and differentiable in t for all s .

The procedure to find the vector field is described as follows. First, we find the closest point $\mathbf{x}^*(\mathbf{x}, t)$ in the curve:

$$\begin{aligned} s^*(\mathbf{x}, t) &= \arg \min_s \|\mathbf{x} - \mathbf{r}(s, t)\|^2, \\ \mathbf{x}^*(\mathbf{x}, t) &= \mathbf{r}(s^*(\mathbf{x}, t), t). \end{aligned} \quad (2)$$

where $(\mathbf{x}, t)$ is always at the subset $\mathcal{U}$ where $s^*(\mathbf{x}, t)$ is a singleton. If $(\mathbf{x}, t) \notin \mathcal{U}$, one possible solution is to use the value that returns the field closest to the last one computed.

Then we find the Euclidean distance vector $\mathbf{D}(\mathbf{x}, t)$ and scalar $D(\mathbf{x}, t)$ given by (3).

$$\begin{aligned} \mathbf{D}(\mathbf{x}, t) &= \mathbf{x} - \mathbf{x}^*(\mathbf{x}, t), \\ D(\mathbf{x}, t) &= \|\mathbf{D}(\mathbf{x}, t)\|. \end{aligned} \quad (3)$$

Next, we define the curve's tangent vector $\mathbf{T}(s, t)$ and the closest point's tangent $\mathbf{T}^*(\mathbf{x}, t)$ defined in (4)¹.

$$\begin{aligned}\mathbf{T}(s, t) &= \frac{\partial \mathbf{r}(s, t)}{\partial s}, \\ \mathbf{T}^*(\mathbf{x}, t) &= \mathbf{T}(s^*, t).\end{aligned}\quad (4)$$

An important feature of the tangent vector $\mathbf{T}^*$ is that it is perpendicular to the Euclidean distance vector $\mathbf{D}$, *i.e.* $\mathbf{D}^T \mathbf{T}^* = 0$. This is a fundamental property to prove the field's convergence [1].

In the sequence, we compute the scalar gains $G(\mathbf{x}, t)$, with $k_G > 0$, and $H(\mathbf{x}, t)$ as in

$$\begin{aligned}G(\mathbf{x}, t) &\equiv G(D(\mathbf{x}, t)) = \frac{2}{\pi} \tan^{-1}(k_G D(\mathbf{x}, t)), \\ H(\mathbf{x}, t) &\equiv H(D(\mathbf{x}, t)) = \sqrt{1 - G(D(\mathbf{x}, t))^2}.\end{aligned}\quad (5)$$

The field is formed by the components below:

$$\begin{aligned}\Phi_S(\mathbf{x}, t) &= -G(D(\mathbf{x}, t)) \frac{\mathbf{D}(\mathbf{x}, t)}{D(\mathbf{x}, t)} \\ &\quad + H(D(\mathbf{x}, t)) \mathbf{T}^*(\mathbf{x}, t), \\ \Phi_T(\mathbf{x}, t) &= -\Pi_T(\mathbf{x}, t) \frac{\partial \mathbf{D}(\mathbf{x}, t)}{\partial t},\end{aligned}\quad (6)$$

where Π_T is the null space projection matrix of $\mathbf{T}^*(\mathbf{x}, t)$.

The last step is to compute another scalar gain $\eta(\mathbf{x}, t)$ that will assure a constant speed $\|\Phi(\mathbf{x}, t)\| = v_r$:

$$\eta = -\Phi_S \cdot \Phi_T + \sqrt{(\Phi_S \cdot \Phi_T)^2 + v_r^2 - \|\Phi_T\|^2}.\quad (7)$$

where the variables dependencies from $(\mathbf{x}, t)$ were omitted to shorten the equation.

Finally, the field is given by (8).

$$\Phi(\mathbf{x}, t) = \eta(\mathbf{x}, t) \Phi_S(\mathbf{x}, t) + \Phi_T(\mathbf{x}, t).\quad (8)$$

The vector field given by (8) has interesting properties such as: (i) the trajectories of system (1) converge to the curve $\mathcal{C}(t)$ for any initial conditions $(\mathbf{x}_0, t_0) \in \mathcal{U}$; (ii) there isn't stable or unstable equilibrium points; (iii) it is robust to bounded additive disturbances for the system (1). Formal proofs and more details are found in [1].

We emphasize that the method in [1] considers no self-intersections in the target curve $\mathcal{C}(t)$. This assumption is required because it searches for the global minimum in the Euclidean distance function for the parameter s at a fixed time t . We show in the Appendix Section that it is possible to follow self-intersecting curves if we consider the local minima of s . The approach, inspired by [8], considers an additional state variable that saves the parameter of the curve that is being followed. In addition, one of the presented results in Section III considers a self-intersecting lemniscate curve.

¹In [1], s is considered as the arc length of the curve. Thereby, the tangent vectors have unit norm. In practice, if s is not the arc length, a normalization can be applied to the tangent vectors.

B. A simple way to circumnavigate an obstacle

In this subsection, we will show how to use the vector field reviewed in Section II-A to circumnavigate an obstacle while keeping a fixed distance from it. The original vector field is computed using the closest point in the target curve. In order to circumnavigate an obstacle, the closest point of the obstacle is used with the same purpose.

Consider the set of collision points $\mathcal{O}(t)$. To navigate safely, the robot's position $\mathbf{x}$ must be at a minimum distance from any $\mathbf{x}_o \in \mathcal{O}(t) \forall t$.

In II-A, the vector field was computed using the closest point on the path $\mathbf{x}^*$. Now, to apply the vector field in the obstacle-circumnavigation task, we consider the closest point $\mathbf{x}_o^*$ on the obstacle defined by

$$\mathbf{x}_o^*(\mathbf{x}, t) = \arg \min_{\mathbf{x}_o \in \mathcal{O}(t)} \|\mathbf{x} - \mathbf{x}_o\|. \quad (9)$$

We also define the obstacle's closest distance vector and scalar distance as:

$$\begin{aligned}\mathbf{D}_o(\mathbf{x}, t) &= \mathbf{x} - \mathbf{x}_o^*(\mathbf{x}, t), \\ D_o(\mathbf{x}, t) &= \|\mathbf{D}_o(\mathbf{x}, t)\|.\end{aligned}\quad (10)$$

The goal is to make the robot circumnavigate the obstacle at a fixed distance $\lambda > 0$. Therefore, to obtain the convergence component, we subtract from $\mathbf{D}_o(\mathbf{x}, t)$ a component with norm λ and in the same direction. Let $\mathbf{D}_\lambda(\mathbf{x}, t)$ be the convergent vector defined by

$$\mathbf{D}_\lambda(\mathbf{x}, t) = \mathbf{D}_o(\mathbf{x}, t) - \lambda \frac{\mathbf{D}_o(\mathbf{x}, t)}{D_o(\mathbf{x}, t)}.\quad (11)$$

Now, consider the equidistant set from the obstacles as in

$$\mathcal{S}_o = \{\mathbf{x} \in \mathbb{R}^n : D_o(\mathbf{x}, t) = \lambda\}.\quad (12)$$

Note that, for the case $n = 2$, the set $\mathcal{S}_o$ is a uni-dimensional set, thus it is equivalent to a path. In this case, the obstacle will be circumnavigated in a defined path. For $n > 2$, the set $\mathcal{S}_o$ is a hyper-surface around the obstacle. In this second case, the path that the obstacle will be circumnavigated is not defined. The method will only ensure that the robot will keep at a distance λ from the obstacle and move along it.

Figure 1 illustrates our definitions for the two-dimensional case. The gray region represents the set $\mathcal{O}$. The green dot is the point $\mathbf{x}_o^*(\mathbf{x}, t)$, and the red vector is $\mathbf{D}_o(\mathbf{x}, t)$. The set $\mathcal{S}_o$ is depicted by the blue path, which is at a distance λ from the set $\mathcal{O}$.

In the case of fields in two dimensions, $n = 2$, there is only one possible direction for the tangent component. In the case $n > 2$ there is more than one possible direction since all we need is that the tangent component is perpendicular to the distance vector. We propose an additional vector field to choose among the possible directions. Let $\mathcal{A}(\mathbf{x}, t)$ be this additional auxiliary field. Then, the tangent component of the field to circulate the obstacle, let us call it $\mathbf{T}_\lambda$, is computed by removing the component of $\mathcal{A}(\mathbf{x}, t)$ in the direction of $\nabla \mathbf{D}_\lambda = \frac{\mathbf{D}_\lambda}{D_\lambda}$ and normalizing the result. This tangent vector is given by

$$\mathbf{T}_\lambda(\mathbf{x}, t) = \frac{\Pi_{\nabla \mathbf{D}_\lambda}(\mathbf{x}, t) \mathcal{A}(\mathbf{x}, t)}{\|\Pi_{\nabla \mathbf{D}_\lambda}(\mathbf{x}, t) \mathcal{A}(\mathbf{x}, t)\|}, \quad (13)$$

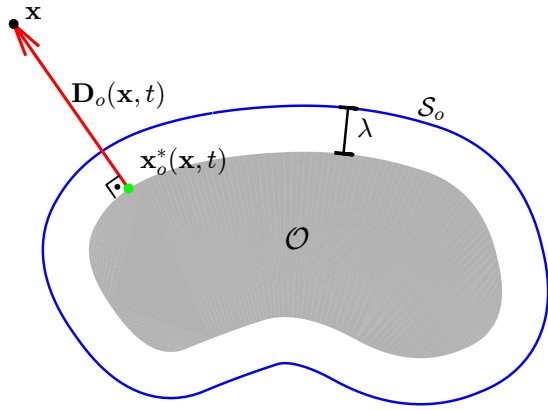


Fig. 1. Illustration of definitions. Obstacle set $\mathcal{O}$ in gray, distance vector $\mathbf{D}_o(\mathbf{x}, t)$ in red, and equidistant set S_o in blue.

in which $\Pi_{\nabla D_\lambda}(\mathbf{x}, t)$ is the projector on the null space of ∇D_λ . Vector $\mathbf{T}_\lambda(\mathbf{x}, t)$ is basically a normalized version of $\mathcal{A}(\mathbf{x}, t)$ without its component in the direction of $\mathbf{D}_o(\mathbf{x}, t)$. For now, we assume that $\mathcal{A}(\mathbf{x}, t)$ is such that $\Pi_{\nabla D_\lambda}(\mathbf{x}, t)\mathcal{A}(\mathbf{x}, t)$ never vanishes.

Figure 2 illustrates the strategy to obtain the vector $\mathbf{T}_\lambda$ in (13) for the three-dimensional case. The internal dark sphere represents an obstacle, while the blue spherical shell represents the set S_o . Any vector $\mathbf{T}_\lambda$ in the gray plane is valid for circulating the obstacle since they are all perpendicular to the vector ∇D_λ , in blue. By considering the auxiliary vector $\mathcal{A}$, in red, we can choose one of these possible tangent vectors. In the image, the vector $\mathbf{T}_\lambda$, in green, corresponds to the projection of $\mathcal{A}$ in the normal plane followed by normalization.

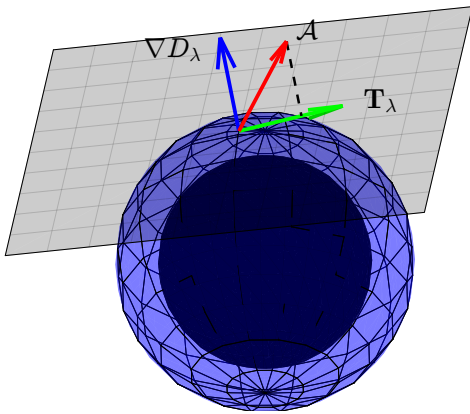


Fig. 2. Illustration of the projection of the auxiliary field $\mathcal{A}(\mathbf{x}, t)$ used to compute a tangent vector $\mathbf{T}_\lambda(\mathbf{x}, t)$.

Now, we are ready to define the obstacle field. Let $\Psi(\mathbf{x}, t)$ be the obstacle field, $\Psi_S(\mathbf{x}, t)$ be its static component with unit norm and $\Psi_T(\mathbf{x}, t)$ be its time feed-forward component.

$\Psi_S(\mathbf{x}, t)$ and $\Psi_T(\mathbf{x}, t)$ are given by

$$\begin{aligned}\Psi_S(\mathbf{x}, t) &= -G(D_\lambda(\mathbf{x}, t)) \frac{\mathbf{D}_\lambda(\mathbf{x}, t)}{D_\lambda(\mathbf{x}, t)} \\ &\quad + H(D_\lambda(\mathbf{x}, t)) \mathbf{T}_\lambda(\mathbf{x}, t), \\ \Psi_T &= -\Pi_{T_\lambda} \frac{\partial \mathbf{D}_\lambda}{\partial t},\end{aligned}\quad (14)$$

in which Π_{T_λ} is the projector on the null space of $\mathbf{T}_\lambda$. The field $\Psi(\mathbf{x}, t)$ is defined by

$$\Psi(\mathbf{x}, t) = \eta_o(\mathbf{x}, t) \Psi_S(\mathbf{x}, t) + \Psi_T(\mathbf{x}, t), \quad (15)$$

in which $\eta_o(\mathbf{x}, t)$ is a scalar gain computed to assure $\|\Psi(\mathbf{x}, t)\| = v_r$ just like $\eta(\mathbf{x}, t)$ in (7) for the field $\Phi(\mathbf{x}, t)$.

Figure 3 depicts, for the time-invariant case and $n = 2$, two fields that circumnavigate an obstacle. On the left we show the auxiliary fields $\mathcal{A}(\mathbf{x})$, and on the right, we show the associated field $\Psi(\mathbf{x})$ that circumnavigates the gray obstacle. In the top example, the field $\mathcal{A}(\mathbf{x})$ is constant. In this case, the field $\Psi(\mathbf{x})$ circumnavigates the obstacle clockwise in the top and counter-clockwise in the bottom. In the bottom example, the field $\mathcal{A}(\mathbf{x})$ is a circulating field, thus, the resulting field Ψ always circumnavigates the obstacle in the same direction. In these two-dimensional cases, the only influence of field $\mathcal{A}$ is in the sense of direction of the field Ψ , since there is only one direction perpendicular to the convergence component.

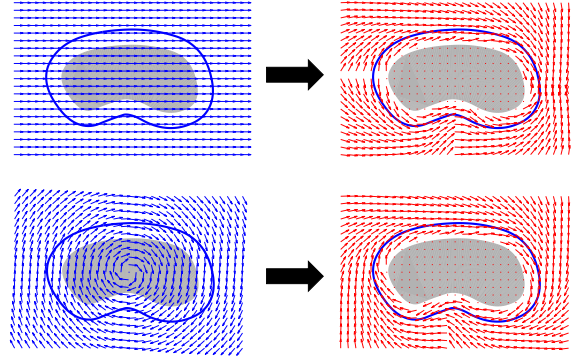


Fig. 3. Two examples (top and bottom) of fields that circumnavigate the gray obstacle. On the left we show the auxiliary field $\mathcal{A}(\mathbf{x})$ that originated the fields $\Psi(\mathbf{x})$ on the right.

In general, the field Ψ may be prone to singularities and local minimum, as in the top example of Figure 3. To solve these problems, one possible approach is to circulate each obstacle always in the same direction, as in the bottom example of Figure 3. For the two-dimensional case, one simple solution to always circulate each obstacle in the same direction is to obtain $\mathbf{T}_\lambda(\mathbf{x}, t)$ by rotating $\nabla D_\lambda(\mathbf{x}, t)$ 90° clockwise or counter-clockwise. The closest obstacle vector can be obtained only by using local data and, in this case, the entire obstacle field from (15) can be computed only in the local frame, removing the necessity of global localization.

Figure 4 illustrates the method in a two-dimensional scenario, where the robot is represented as a light gray circle and has a range sensor illustrated with red lines. The goal is to follow the dark gray obstacle at a fixed distance λ ,

i.e., follow the uni-dimensional set $\mathcal{S}_o$, in blue. The closest obstacle point is locally measured by the range sensor and has angle φ_m and magnitude δ_m , both expressed in the local frame. The closest obstacle vector $\mathbf{D}_o$ is obtained only with local data in (16). Henceforth, the entire obstacle field can be obtained only with local information.

$$\mathbf{D}_o^{body} = -\delta_m [\cos(\varphi_m) \quad \sin(\varphi_m)]^T. \quad (16)$$

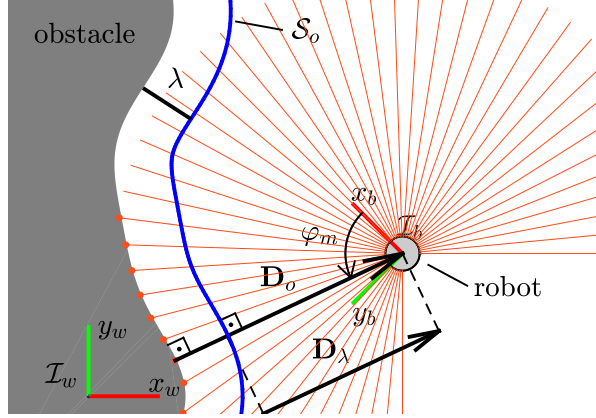


Fig. 4. Example of the obstacle field Ψ for the simple two-dimensional case that can be computed only with local information. The robot is represented as a gray circle with a range sensor represented by red lines. The obstacle is illustrated in dark gray at the left. The set $\mathcal{S}_o$ that keeps a distance λ from the obstacle is represented in a blue line.

C. Obstacle avoidance incorporation

At this point, we have reviewed the general vector field $\Phi(\mathbf{x}, t)$ formulation from [1] to follow a given curve $\mathcal{C}(t)$ in II-A and we also adapted this formulation to circumnavigate obstacles in II-B by computing the field $\Psi(\mathbf{x}, t)$. The goal in this subsection is to combine $\Phi(\mathbf{x}, t)$ and $\Psi(\mathbf{x}, t)$ to generate a new field $\mathcal{F}(\mathbf{x}, t)$ capable of guiding the system (1) to a time-varying path $\mathcal{C}(t)$ while avoiding locally sensed obstacles.

The idea of the navigation approach is to follow the vector field to the original path when there is no imminent collision with any obstacle. When an obstacle is in the way, the navigation approach will be the circumnavigation of the obstacle. By imminent collision, we mean a situation that respects two conditions. First, the direction of the original curve field points in the direction that makes the distance to the obstacle decrease. Second, the distance between the state $\mathbf{x}$ and the closest obstacle is less than a threshold D_{in} . If these conditions are met, the navigation uses the obstacle field. The robot will attempt to circumnavigate the obstacle at a distance λ from the closest obstacle. It is necessary that the value of D_{in} is such that $\lambda < D_{in}$.

In order to compute the obstacle field $\Psi(\mathbf{x}, t)$ we use the auxiliary field $\mathcal{A}(\mathbf{x}, t)$ as the original curve field $\Phi(\mathbf{x}, t)$, $\mathcal{A}(\mathbf{x}, t) = \Phi(\mathbf{x}, t)$. With this strategy, the system will attempt to circumnavigate the obstacle as close as possible to the original field that converges to the target curve.

To avoid abrupt transitions between the fields, we also consider an intermediate state when the distance between $\mathbf{x}$ and the set $\mathcal{O}(t)$ is greater than D_{in} but smaller than a distance $D_{in}^0 > D_{in}$. In this state, the field will be defined as a combination of the fields. To compute this composite field, consider the parameter θ defined as

$$\theta(\mathbf{x}, t) \equiv \theta = \frac{D_o(\mathbf{x}, t) - D_{in}}{D_{in}^0 - D_{in}}. \quad (17)$$

Finally, the navigation field $\mathcal{F}(\mathbf{x}, t)$ is defined by

$$\mathcal{F} = \begin{cases} \Phi, & \mathbf{D}_o^T \Phi \geq 0 \text{ or } D_o > D_{in}^0, \\ \Psi, & \mathbf{D}_o^T \Phi < 0, D_o < D_{in}, \\ v_r \frac{\theta \Phi + (1-\theta) \Psi}{\|\theta \Phi + (1-\theta) \Psi\|}, & \text{otherwise,} \end{cases} \quad (18)$$

in which the dependencies on $\mathbf{x}$ and t were omitted.

Figure 5 illustrates the vector field defined in (18). In the figure, the dark shapes are the obstacles. The field depicted in black converges to the curve, also in black, while avoiding the obstacles. In the blue region, the original field Φ is active. In the green region, we have the composite transition field, and in the yellow region, the field Ψ is active. In the top right, we see that the example trajectory (in red) deviates from an obstacle that is far away from the target curve. At the bottom, we see that the trajectory also deviates from an obstacle on the original path. In this case, the trajectory is forced to leave the path. This effect may also happen when the obstacle is close to the curve (top left).

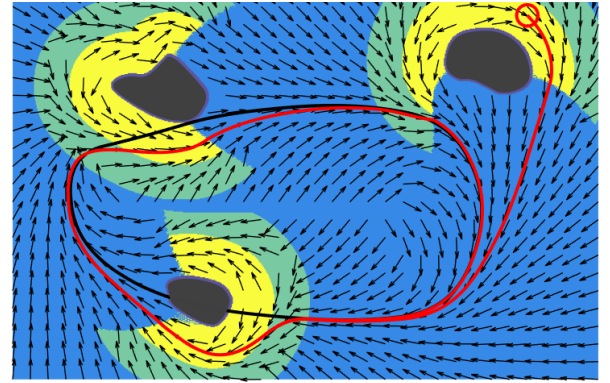


Fig. 5. Example of the vector field defined in (18). The red trajectory attempts to circulate the path in black while deviating from the obstacles in dark gray. The regions in blue, yellow and green show which of the three conditions in equation (18) is active.

III. RESULTS AND DISCUSSIONS

In this section, we present the results and discussions of simulations with the widely used Robot Operating System (ROS) [20] and real-world experiments, also based on ROS architecture, using a grounded wheeled robot and a UAV quadrotor.

A. Computational simulations

To validate the obstacle field proposed in Subsection II-B, we simulated a differential drive robot in Stage simulator together with ROS (wiki.ros.org/stage) for different values

of λ . For the low-level controller, we used a static-feedback linearization approach [2], [21] that was computed in the local frame along with the field itself. Figure 6 shows the robot trajectories in colored lines and the obstacle in gray. The robot successfully followed the generic-shaped obstacle for different values of λ . At the bottom, when λ is small enough, the robot is allowed to navigate between the two obstacles. When it is not, the field circumnavigates the obstacles without entering the in-between area and therefore respecting the λ restriction. Also, on the left, the robot was able to circumnavigate the obstacle even with sharp edges and non-convex shapes.

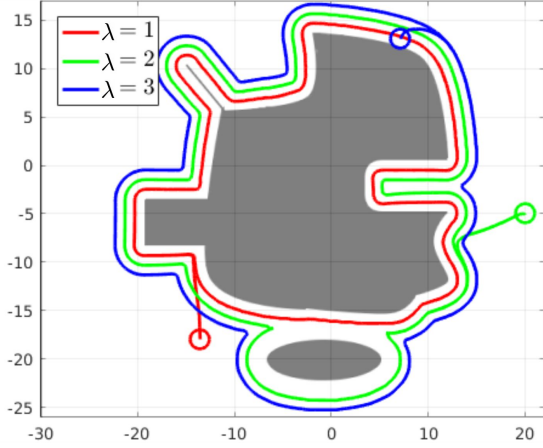


Fig. 6. Results of obstacle field applied to a differential drive robot using StageROS simulator. The trajectories are shown in colored lines for different values of λ , while the circumnavigated obstacle is shown in gray.

To perform simulations using ROS to follow curves while deviating from obstacles, we implemented a library called *vectorfield_stack*. The library is a collection of ROS packages developed in Python. It is available at github.com/adrianomcr/vectorfield_stack.

An example using a UAV quadcopter to explain the library is presented next. For this, the considered curve is described by the following parametric equation:

$$\mathbf{r}(s, t) \equiv \mathbf{r}(s) = \begin{bmatrix} 2 \cos(s) & 2 \sin(s) & 0.2 \cos(6s) \end{bmatrix}^T, \quad (19)$$

in which $0 \leq s \leq 2\pi$. The parameters associated to the obstacle avoidance are: $\lambda = 0.4$ m, $D_{in} = 0.55$ m and $D_{in}^0 = 0.7$ m.

Figure 7 shows the curve $\mathcal{C}$ in green and the obstacles in red. The UAV's trajectory is shown in gray. After the simulation starts, the vehicle deviates from the first obstacle. After the vehicle has already approached the curve, the trajectory deviates from two obstacles. In both cases, the deviated trajectory goes in a direction that is closer to the original vector field. One of the obstacles is circumnavigated from below, while the other is from above. The white dot is the closest point in the obstacle set $\mathcal{O}$, which is used to compute the obstacle field. When the robot is already at the curve and starts to circumnavigate an obstacle, the distance to the curve increases. However, after the obstacle is circumnavigated, the distance to the curve decreases again.

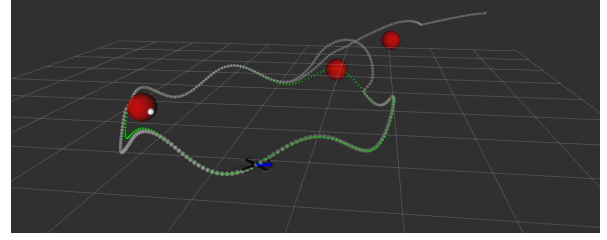


Fig. 7. Simulation result of a UAV quadcopter following a vector field while deviating from obstacles.

Figure 8 shows the distance (to the curve $\mathcal{C}$) function D in blue and the distance (to the obstacle set $\mathcal{O}$) function D_o in red. Initially, D is approximately 3.8 m, and rapidly decreases to almost 0. When D_o becomes smaller than D_{in}^0 (yellow line), and the quadcopter starts to circumnavigate the obstacle, the distance D increases. However, as soon as the robot returns to follow the original vector field, the distance D decreases again. While the obstacle is circumnavigated, we observe that D_o remains approximately constant at the value of $D_o = \lambda$, the green line. Note also that D_o approaches λ from above, thus, if λ is set properly, the vehicle will never get sufficiently close to the obstacle so that a collision occurs. To control the non-linear dynamics of the UAV quadcopter the library *vectorfield_stack* uses the controller proposed in [4].

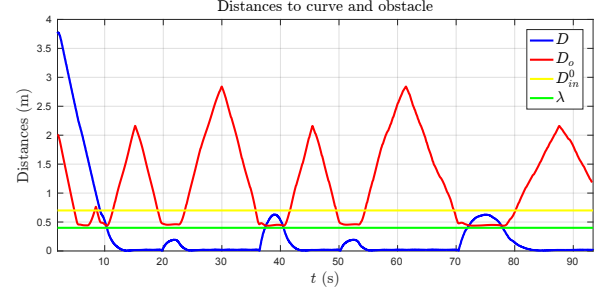


Fig. 8. Distance function in the UAV quadcopter simulation using the *vectorfield_stack* library. The distance to the curve (D) is in blue and the distance to the obstacle set (D_o) is in red. The yellow and green lines represent D_{in}^0 and λ , respectively.

A video with several simulations using the library *vectorfield_stack* is available at youtu.be/9gtiL80KUsc.

B. Wheeled robot experiments

In a real-world experiment with a wheeled robot developed by the Vale Institute of Technology (ITV, itv.org/en/home), EspeleoRobô [22] was controlled to follow a lemniscate curve. In this case, the localization was performed with the gmapping algorithm [23], fed with the data of a Hokuyo sensor, in a ROS-based system. While the robot was following the curve, traffic cones were placed in different positions of the workspace to evaluate the system's efficiency in deviating from them. In this experiment, we used the strategy described in the Appendix Section to allow the consideration of the self-intersecting curve.

Figure 9 shows, from top to bottom, three different stages of the experiment's execution. On the left, we show a frame of a recording of the experiment corresponding to the plot on the right. In the plots, the curve is shown in black, and the robot's past trajectory is in red. The cones are represented by orange circles. Note that, at the top, with any obstacles, the trajectory converges to the lemniscate path. In the middle, two cones were placed in the robot's path. The first one was circumnavigated clockwise, while the second one was counter-clockwise. The distance to the curve increases when the robot is deviating from the obstacles. However, it returns to decrease as the obstacles are left behind. To control the non-linear model of EspeleoRobô we use a static-feedback linearization approach [2], [21].

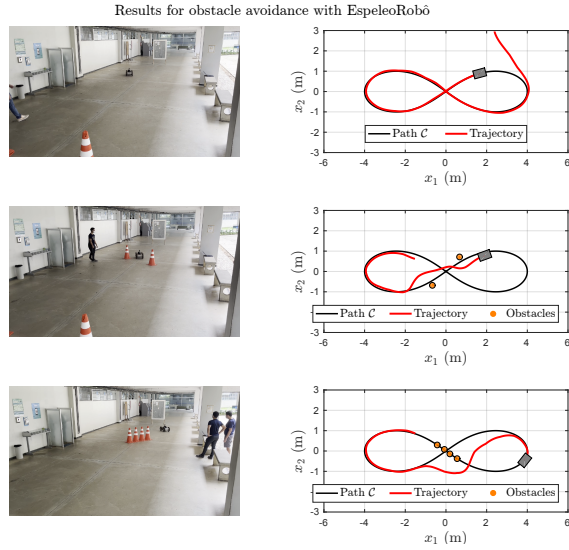


Fig. 9. From top to bottom on the left, three frames of the recording of the experiment of the EspeleoRobô following a lemniscate curve while deviating from obstacles. On the right we show respective plots of the robot's past trajectory in red, with the curve in black and the obstacles in orange.

A video with the recording of the experiment in which the EspeleoRobô followed the lemniscate and deviated from the obstacles is available at youtu.be/hQuCXnLEOd8.

C. UAV robot indoor experiments

In an indoor UAV quadrotor robot experiment, a low-cost drone named ESPcopter (espcopter.com), was controlled to follow an ellipsoid curve while deviating from a styrofoam ball attached to a stick. The localization was performed using an OptiTrack Motion Capture system along with the software Motive (optitrack.com/software/motive). The system counts with 8 Prime 41 cameras placed in a space of 36 m², providing measurements at 180 Hz with an error of less than 1 mm. As depicted in Figure 10 the drone is connected to a computer's hotspot via Wi-Fi 2.4GHz. The controller runs in a ROS framework in this computer that is receiving the Motive data via ethernet connection.

Figure 11 shows three frames of ESPcopter experiment on the left. From top to bottom, the frames follow a time sequence. ESPcopter is initially following the target

curve, then deviates from the obstacle by doing the circumnavigation, and finally returns to follow the curve. Each frame corresponds to the visualization chart in RViz on the right. A complete video with the recording of this experiment is available at youtu.be/DxIbqnbUsrQ.

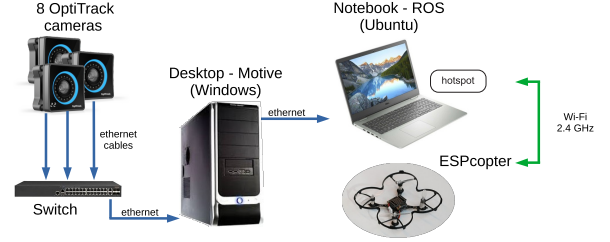


Fig. 10. Setup for the ESPcopter experiment.

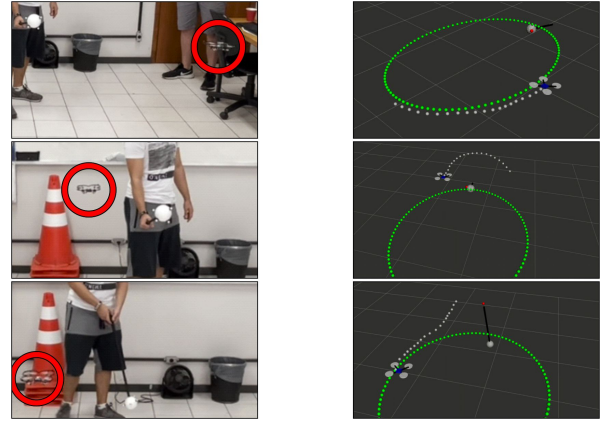


Fig. 11. On the left, three frames of ESPcopter following a curve and deviating from the obstacle by circumnavigating it. The ESPcopter is highlighted with a red circle. On the right, visualization charts in RViz correspondent to each left frame.

Figure 12 shows the result for this experiment. It is very similar to the ROS simulation result presented in Figure 8, with D in blue and D_o in red. The same effects of the simulations happen in this experiment, but also unknown and unmodeled noises act in this scenario for both D and D_o . The non-linear dynamics were controlled with a C++ implementation of [4].

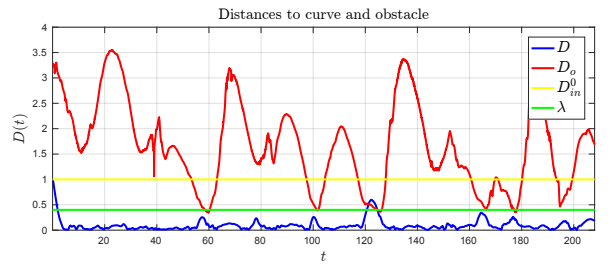


Fig. 12. Distance function in the ESPcopter experiment. The distance to the curve (D) is in blue and the distance to the obstacle set (D_o) is in red. The yellow and green lines represent D_{in}^0 and λ , respectively.

IV. CONCLUSIONS AND FUTURE WORKS

The main contribution of this work is the extension of the method in [1] to allow obstacle avoidance by circumnavigating local sensed obstacles when there is an imminent collision in the original trajectory to the goal curve. Therefore, generating a collision-free vector field with the advantages of [1]. ROS-based computational simulations and real-world experiments were presented to validate our proposed extension. The proposed scheme uses a reactive circumnavigation approach which is different from deliberative and reactive repulsive approaches.

As mentioned in Section II, when choosing the auxiliary field $\mathcal{A}(\mathbf{x}, t)$, the obstacle field may be prone to singularities and local minima. A simple solution to deal with this problem is proposed for the two-dimensional case, where instead of using the auxiliary field to find the tangent vector, a simple rotation can be applied to the distance vector.

To continue our development, we intend to deal with the local minima and singularities that may happen in the obstacle field, as discussed above. Also, we intend to formalize the theory that allows the field to be applied to self-intersecting curves, as shown in the Appendix Section below and presented in the wheeled robot experiment.

APPENDIX

As mentioned in the Vector Field Section and as used in the EspeleoRobô experiment in Results Section, it is possible to use the Vector Field method to follow self-intersecting curves.

Our approach to following these types of curves is the consideration of a local minima of the optimization problem in (2). To do that, we save the parameter associated with the closest point on the curve $\mathbf{r}(s, t)$. Let this memory variable be s_m . Instead of looking for the closest point on the entire curve, we will search for the closest point only locally. We consider a search domain of length $2\delta_s > 0$. This approach can be implemented by replacing s^* in (2) by (20).

$$s^*(\mathbf{x}, t, s_m) = \arg \min_{s_m - \delta_s < s < s_m + \delta_s} \|\mathbf{x} - \mathbf{r}(s, t)\|^2. \quad (20)$$

When the implementation of the vector field departs from a curve represented by a sequence of points, one can simply look for the closest one among the neighbors of the current closest point. We emphasize that this methodology is not formal. A better mathematical formulation of this idea is a topic for future research.

REFERENCES

- [1] A. M. C. Rezende, V. M. Gonçalves, and L. C. A. Pimenta, "Constructive Time-Varying Vector Fields for Robot Navigation," *IEEE Transactions on Robotics*, pp. 1–16, 2021.
- [2] V. M. Gonçalves, L. C. A. Pimenta, C. A. Maia, B. C. O. Dutra, and G. A. S. Pereira, "Vector Fields for Robot Navigation Along Time-Varying Curves in n -Dimensions," *IEEE Transactions on Robotics*, vol. 26, pp. 647–659, Aug. 2010.
- [3] V. M. Gonçalves, L. C. A. Pimenta, C. A. Maia, G. A. S. Pereira, B. C. O. Dutra, N. Michael, J. Fink, and V. Kumar, "Circulation of curves using vector fields: Actual robot experiments in 2D and 3D workspaces," in *2010 IEEE International Conference on Robotics and Automation*, IEEE, May 2010.

- [4] A. M. C. Rezende, V. M. Gonçalves, A. H. D. Nunes, and L. C. A. Pimenta, "Robust quadcopter control with artificial vector fields," in *2020 IEEE International Conference on Robotics and Automation (ICRA)*, IEEE, May 2020.
- [5] L. A. A. Pereira, A. H. D. Nunes, A. M. C. Rezende, V. M. Gonçalves, G. V. Raffo, and L. C. A. Pimenta, "Collision-free vector field guidance and MPC for a fixed-wing UAV," in *2021 IEEE International Conference on Robotics and Automation (ICRA)*, IEEE, May 2021.
- [6] D. A. Lawrence, E. W. Frew, and W. J. Pisano, "Lyapunov Vector Fields for Autonomous Unmanned Aircraft Flight Control," *Journal of Guidance, Control, and Dynamics*, vol. 31, pp. 1220–1229, Sept. 2008.
- [7] D. Nelson, D. Barber, T. McLain, and R. Beard, "Vector Field Path Following for Miniature Air Vehicles," *IEEE Transactions on Robotics*, vol. 23, pp. 519–529, June 2007.
- [8] W. Yao, H. G. de Marina, B. Lin, and M. Cao, "Singularity-Free Guiding Vector Field for Robot Navigation," *IEEE Transactions on Robotics*, vol. 37, pp. 1206–1221, Aug. 2021.
- [9] V. M. Gonçalves, C. A. Maia, G. A. S. Pereira, and L. C. d. A. Pimenta, "Navegação de robôs utilizando curvas implícitas," *Sba: Controle & Automação Sociedade Brasileira de Automatica*, vol. 21, pp. 43–57, 2010.
- [10] O. Khatib, "Real-Time Obstacle Avoidance for Manipulators and Mobile Robots," in *Autonomous Robot Vehicles*, pp. 396–404, Springer New York, 1986.
- [11] H. Choset, K. Lynch, S. Hutchinson, G. Kantor, W. Burgard, L. Kavraki, and S. Thrun, *Principles of Robot Motion: Theory, Algorithms, and Implementations*. MIT Press, May 2005.
- [12] A. Jahn and L. C. Pimenta, "Sampling Based Path Planning and Vector Fields for Curve Tracking by UAVs," in *2016 XIII Latin American Robotics Symposium and IV Brazilian Robotics Symposium (LARS/SBR)*, IEEE, Oct. 2016.
- [13] I. Ko, B. Kim, and F. C. Park, "Randomized path planning on vector fields," *The International Journal of Robotics Research*, vol. 33, pp. 1664–1682, Oct. 2014.
- [14] G. A. S. Pereira, S. Choudhury, and S. Scherer, "A framework for optimal repairing of vector field-based motion plans," in *2016 International Conference on Unmanned Aircraft Systems (ICUAS)*, IEEE, June 2016.
- [15] G. A. S. Pereira and E. J. R. Freitas, "Navigation of Semi-autonomous Service Robots Using Local Information and Anytime Motion Planners," *Robotica*, vol. 38, pp. 2080–2098, Jan. 2020.
- [16] M. H. Tanveer, C. T. Recchiuto, and A. Sgorbissa, "Analysis of path following and obstacle avoidance for multiple wheeled robots in a shared workspace," *Robotica*, vol. 37, pp. 80–108, Aug. 2018.
- [17] M. Iacono and A. Sgorbissa, "Path following and obstacle avoidance for an autonomous UAV using a depth camera," *Robotics and Autonomous Systems*, vol. 106, pp. 38–46, Aug. 2018.
- [18] J. P. Wilhelm and G. Clem, "Vector Field UAV Guidance for Path Following and Obstacle Avoidance with Minimal Deviation," *Journal of Guidance, Control, and Dynamics*, vol. 42, pp. 1848–1856, Aug. 2019.
- [19] W. Yao, B. Lin, and M. Cao, "Integrated Path Following and Collision Avoidance Using a Composite Vector Field," in *2019 IEEE 58th Conference on Decision and Control (CDC)*, IEEE, Dec. 2019.
- [20] M. Quigley, K. Conley, B. Gerkey, J. Faust, T. Foote, J. Leibs, R. Wheeler, A. Y. Ng, et al., "ROS: an open-source robot operating system," in *ICRA workshop on open source software*, vol. 3, p. 5, Kobe, Japan, 2009.
- [21] H. K. Khalil, *Nonlinear systems*, vol. 3. Prentice hall Upper Saddle River, NJ, 2002.
- [22] H. Azpúrua, A. Rezende, G. Potje, G. P. da Cruz Júnior, R. Fernandes, V. Miranda, L. W. de Resende Filho, J. Domingues, F. Rocha, F. L. M. de Sousa, L. G. D. de Barros, E. R. Nascimento, D. G. Macharet, G. Pessin, and G. M. Freitas, "Towards Semi-autonomous Robotic Inspection and Mapping in Confined Spaces with the EspeleoRobô," *Journal of Intelligent & Robotic Systems*, vol. 101, Mar. 2021.
- [23] Y. Abdelrasoul, A. B. S. H. Saman, and P. Sebastian, "A quantitative study of tuning ROS gmapping parameters and their effect on performing indoor 2d SLAM," in *2016 2nd IEEE International Symposium on Robotics and Manufacturing Automation (ROMA)*, IEEE, Sept. 2016.